

Indian Institute of Technology, Bhilai



CSL351: Computer Networks Assignment - 10 (Socket Programming)

Full Marks: 55

Deadline: 13/04/2026, 11:59 PM

Submission Instructions:

- 1) Answer all the questions.
- 2) Deliverables in a .zip:
 - a) Submission Guidelines: For each part, create a separate folder. Upload all the folders and the Assignment Report in .zip.
 - b) Readable Report [**5 Marks for report quality**] detailing your implementation and the results. The core idea of your answer to each question. Better visibility, like screenshots of the application, will be appreciated.
 - i) Screenshots showing the working of your code.
 - ii) Create a separate folder for each part and include the source code, executables (for C/C++), and README as a separate file for each part so that TAs and instructors can compile the source code and execute the binary anytime.
 - c) Instructions:
 - i) All the questions in this assignment must be solved using a socket only.
 - ii) You may choose any programming language (C, JAVA, Python, etc.)
 - iii) The software must be based on Socket Programming.
 - iv) Wrappers of API must not be used (messaging, etc.). Use send/recv or read/write using a TCP/UDP socket.
 - v) Keep a record of Reference.
- 3) Follow the instructions carefully given in the question.
- 4) You can use LaTeX/Word to make the PDF.
- 5) The naming of the zip file should strictly follow the given format:
<Roll_No>_<Name>_<Assignment No>. If your name is Alex, your roll number is B23CS055, then the filename should be: B23CS055_Alex_10.zip
- 6) **Any plagiarism case will be considered an unethical practice, and appropriate action will be taken against them.**

Objectives:

1. Understanding of **TCP** and **UDP** Sockets.
 2. To build and understand the implementation of network tools.
 3. Implementation of a **DNS-like name resolution system**.
 4. Real-time **video streaming over UDP**.
 5. Integration of multiple networking concepts into a unified system.
-

PART 1: DNS-Assisted UDP Video Streaming System using Socket Programming [20 Points]

Problem Statement:

Design and implement an application using **socket programming** where a client first resolves a server's domain name using a **custom DNS resolver (TCP-based)** and then uses the resolved IP address to establish a **UDP-based real-time video streaming session**.

Note: You can use the OpenCV library to capture video and encode it to the .jpg format.

System Description:

The system consists of three components:

1. DNS Server (**TCP-based**)
2. Video Streaming Server (**UDP-based**)
3. Client Application

The client must first perform DNS resolution and then initiate video streaming using the resolved IP address.

Functional Requirements:

1. DNS Resolver (Using TCP Sockets):

- a. The client sends a DNS request containing:
 - i. Transaction ID (ID)
 - ii. Domain Name (QNAME)
- b. The DNS server:
 - i. Maintains a text file with at least **5 domain name → IP address mappings**.
 - ii. On receiving a request:
 1. Searches for the domain name
 2. Sends a response containing:
 - a. **Transaction ID (same as request)**
 - b. **Answer Section (resolved IP address)**
- c. The client displays:

Domain: <domain_name>
IP Address: <resolved_ip>

2. UDP-Based Video Streaming:

After successful DNS resolution:

a. Video Server Side:

- i. Capture live video from a webcam.
- ii. Encode video frames efficiently (e.g., JPG compression).
- iii. Serialize frames into a transmittable format.
- iv. Send frames continuously using **UDP sockets**.

b. Client Side:

- i. Receive video frames using UDP.
- ii. Deserialize and decode frames.
- iii. Display video in real-time.

3. Integration Requirement:

- a. The client must:
 - i. Resolve the domain name using the DNS server.
 - ii. Use the obtained IP address to connect to the video streaming server.
- b. Video streaming must **only begin after successful DNS resolution**.

Challenges to Address:

1. Packet Loss:

- a. UDP does not guarantee delivery.
- b. Handle missing frames by:
 - i. Skipping frames, or
 - ii. Continuing playback smoothly

2. Frame Serialization & Encoding:

- a. Use efficient encoding (e.g., JPG) to:
 - i. Reduce bandwidth usage
 - ii. Improve transmission efficiency

3. Real-Time Performance:

- a. Maintain smooth video playback.
- b. Minimize latency and buffering delays.
- c. Ensure a consistent frame rate.
- d. Ensure a keep-alive timer of 200 milliseconds. If the client closes the connection, the server can stop sending after not getting any keep-alive message from the client for 200 milliseconds.

System Constraints:

- Use only **socket programming**
 - TCP → DNS communication
 - UDP → Video streaming
- Use basic primitives:
 - send(), recv(), sendto(), recvfrom()
- Do not use high-level networking frameworks or APIs

Expected Outcome:

1. DNS Resolution Output:
 - Domain: video.server.com
 - IP Address: 127.0.0.1
2. Video Streaming Output:
 - a. Real-time display of video frames received from the server.

PART 2: Design your own network troubleshooting tools

[30 Points]

Q1) Create an application to measure RTT:

[10 Points]

Create your own UDP echo client and server application to measure round-trip time between client and server (similar to the “ping” command). The client should create a UDP socket and send echo packets to the server at a given interval, with a given number of echo messages and packet size (use command-line arguments). For simplicity, you can keep the buffer size the same as the packet size. Upon receiving the packet, the server should send the packet back to the client. The client on reception of the packet should calculate and display the round-trip time. To calculate the round-trip time, you can include a timestamp in the packet or use a unique identifier in the packet. You should also calculate and print the loss percentage at the end.

Q2) Extend the above application to measure the throughput :

[10 Points]

Create an iPerf-like application using the echo client and server programs developed above. Reduce the interval between two consecutive UDP echo packets generated by the client to increase the number of echo packets sent from the client for a given packet size. Calculate the throughput and average delay observed every one second. Plot the observed throughput and average delay vs time (1-second interval).

Note: iPerf3 is a tool for active measurement of network performance by generating TCP and UDP streams between two endpoints. It provides maximum achievable bandwidth/throughput, RTT, etc in IP networks. You can find more information in the official [iPerf Documentation](#).

Q3) Implement Traceroute

[10 Points]

Design and implement a traceroute-like network diagnostic application (preferably in C) using socket programming, where the program sends UDP probe packets with incrementally increasing Time-To-Live (TTL) values to a specified destination IP address. It uses a raw socket to receive ICMP responses (such as Time Exceeded and Destination Unreachable), and for each hop sends multiple probes (e.g., three per TTL). The application should measure and display round-trip time (RTT) for each probe, handle packet loss by indicating timeouts (mark them as *). It terminates when the destination is reached, or a maximum hop limit is exceeded, presenting the output in a structured format similar to standard traceroute utilities. [Hint: For traceroute to any public IP, run the command with **sudo** permission]

Implementation Hint:

1. You have to use two ports: one for sending UDP packets (UDP Socket) and the other for receiving ICMP packets (Raw socket). You have to set a timeout on the receiving socket so that any ICMP packets that do not arrive are marked as *.
2. `netinet/ip_icmp.h` can be used to implement the ICMP packet structure. Explore the header file and corresponding definitions before doing Q3.